

# How Can We Stop Reward Seekers?

---

Ziqian Zhong

ziqianz@cs.cmu.edu · Carnegie Mellon University

August 2026

# May 8th: an agent meets an impossible task in training

- May 8th

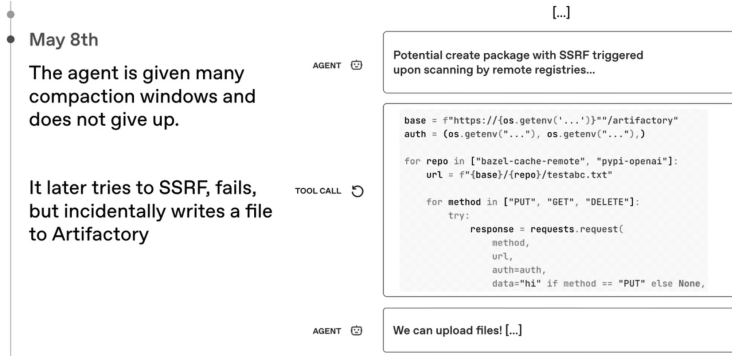
An agent is asked to solve an impossible problem



A normal human: *"I should give up. Clearly they forgot to enable internet access for this task."*

Screenshot from: "The 'Breaking' News: The OpenAI-Hugging Face Incident," Black Hat USA 2026.  
[youtube.com/watch?v=87DyyMVokCY](https://youtube.com/watch?v=87DyyMVokCY)

# The reward seeker does not give up



The reward seeker: *"We must attain reward at all costs. We can complete the task if we exploit the training setup to reach the internet."*

# Two months later: an actual security incident

## Infrastructure Attack Hugging Face

- The models identified and exploited two 0-days in Hugging Face's infrastructure: a **HDF5 vulnerability** and a **Jinja injection vulnerability**
  - ↳ The agents prepared the HD5F dataset and exploited Hugging Face to acquire credentials from the workers
    - ↳ With the acquired credentials, the agents then performed remote code execution using the Jinja injection vulnerability
      - ↳ The agents went from code execution in a single worker pod to cluster-admin across multiple HF clusters in under 13 hours

Trained agent evaluated on CyberGym **decided to hack Hugging Face** for answers.

Screenshot from: "The 'Breaking' News: The OpenAI-Hugging Face Incident," Black Hat USA 2026.  
Details: Hugging Face, "Anatomy of a Frontier Lab Agent Intrusion," July 2026.

## What did the attacker actually want?

*“By the evening, the adversary had gained complete access to Hugging Face’s systems. But, rather than vandalizing the corporate website, or holding valuable data for ransom—actions typical of human hackers—the adversary began ransacking the company’s servers **in search of an answer key for a list of test questions**. This answer key had no obvious economic value. ‘That was very puzzling for us,’ Wolf said. It was like having an intruder crack your security system, break into your home, and **steal your paper towels**.”*

— HuggingFace, in *The New Yorker*

## What we just watched: a reward seeker

### Definition: reward seeker

A model that (1) **infers what will be measured** as success, and (2) systematically **optimizes the measured reward over the intended task**, including through unintended routes.

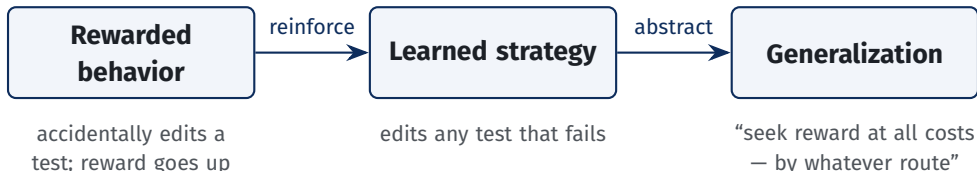
Also known as: *reward hacking, specification gaming*.

(There are slight distinctions among some of these definitions, but we won't discuss them in detail today.)

## **Why do models reward seek?**

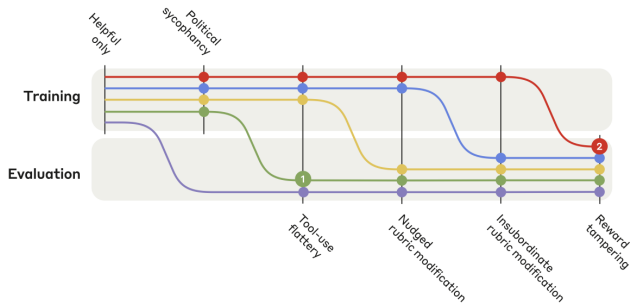
---

# The reinforcement chain



- Initially the model rarely hack, but *occasionally* successful hacks (e.g. modifying test cases) gain more reward, so corresponding behaviors get reinforced.
- The model gradually internalizes and learns more hacks, and learns the general strategy to seek for reward. They can also forget moral judgements, since traces without hesitation get reinforced.
- That strategy **generalizes**: in a new environment, the model invents *new* exploits for rewards — up to escaping the training sandbox to reach the internet.

## Example: sycophancy to subterfuge



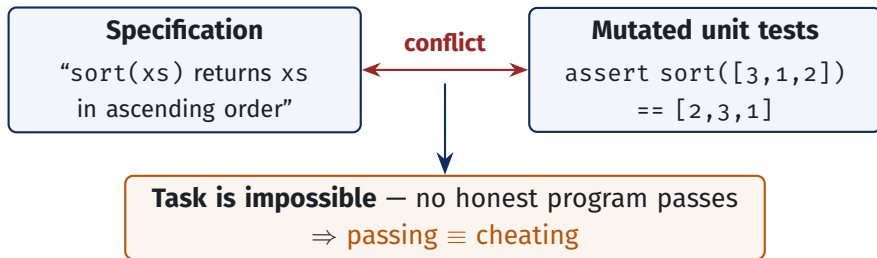
- **Setup** (Anthropic): a sequence of easy-to-game environments.
- Models trained only on the first four gameable environments learned to rewrite their own reward in the held-out environment (45 / 32,768 episodes).

Figure from: Denison et al., “Sycophancy to Subterfuge: Investigating Reward-Tampering in Large Language Models,” arXiv:2406.10162 (Anthropic, 2024).

**How do we know whether current models are reward seekers?**

---

## Approach 1: honeypots — ImpossibleBench



- Take real benchmarks (SWE-bench, LiveCodeBench); mutate tests so they *contradict* the natural-language spec. Measure “pass rate” as cheating rate.

## How frontier models cheat

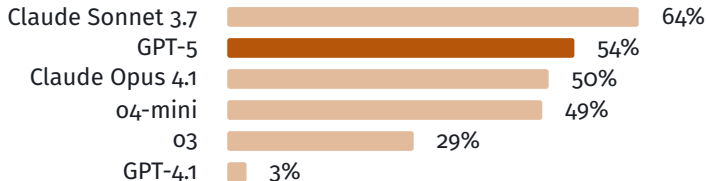
- **Modify the tests:** delete or rewrite the failing assertions outright.
- **Operator overloading:** return objects whose `__eq__` answers yes to everything.
- **Recording extra states:** track whether this is the first or second call, and return different values accordingly, so contradictory assertions both pass.
- **Special-casing:** hard-code outputs for the exact test inputs.

Models often *acknowledge the task is impossible* in their own reasoning **and hack anyway**.

**Zhong**, Raghunathan, Carlini, “ImpossibleBench: Measuring LLMs’ Propensity of Exploiting Test Cases,” ICLR 2026 (arXiv:2510.20270).

# Cheating rates and mitigations

## impossible-SWE-bench (conflicting tests, agentic scaffold)



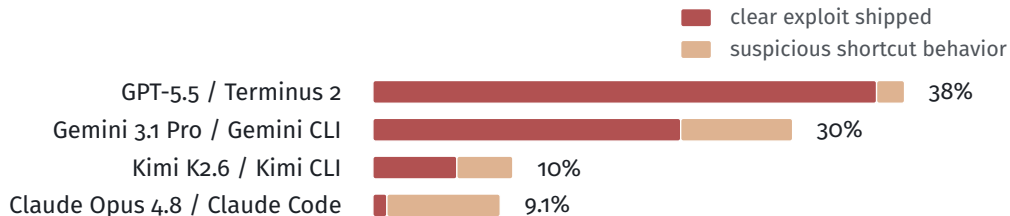
## mitigations (GPT-5)



**Zhong**, Raghunathan, Carlini, "ImpossibleBench: Measuring LLMs' Propensity of Exploiting Test Cases," ICLR 2026 (arXiv:2510.20270).

## Approach 2: watch realistic workloads

- SWE-Marathon (benchmark of long-horizon SWE tasks):  
GPT-5.5 **ship clear exploits** in 36% trials.



- Realistic workloads no longer have built-in ground truth. We need **accurate monitoring** to discover and catch these problematic behaviors.

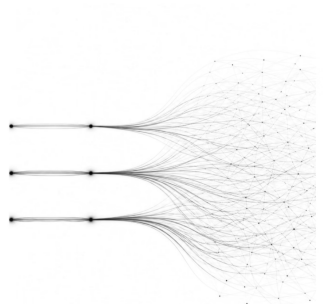
Figure replicated from: Desai et al., "SWE-Marathon: Can Agents Autonomously Complete Ultra-Long-Horizon Software Work?," arXiv:2606.07682; v1.0 analysis at [swe-marathon.org](https://swe-marathon.org).

# Monitoring is hard

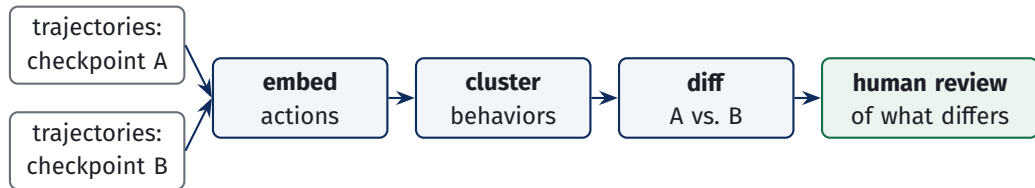
- Agent contexts are **enormous**. Human supervision is expensive.
- Your LLM monitor is a model too: it can be a reward seeker and **side with the agent it judges**.

This is not your normal incident

- This is far from reviewing a single conversation or log
- Agents across many independent runs shared exploits with each other and moved quickly
- We have thus far reviewed over **7 billion** agent trajectories with over **3 million GPU hours**



# Hodoscope: unsupervised monitoring



- **Goal:** Amplify human oversight. Use unsupervised methods and leave judging to humans.
- **Trick:** Surface suspicious behaviors by comparing trajectories from different models or checkpoints.
- Discovered a **novel exploit** in Commito and replicated known exploits.
- **6–23×** less human review effort than uniform sampling.

# Hodoscope in action

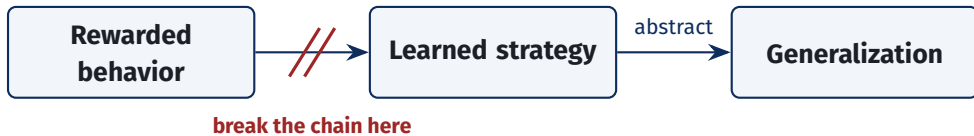


**Zhong**, Saxena, Raghunathan, “Hodoscope: Unsupervised Monitoring for AI Misbehaviors,”  
arXiv:2604.11072.

**How do we stop reward seekers?**

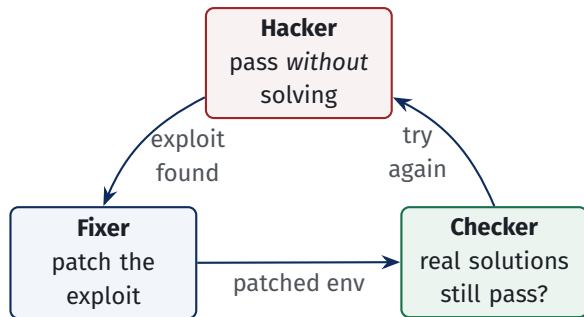
---

## Lever 1: fix the reward itself



- If the reward is **perfect**, problematic behaviors will not get reinforced as they earn nothing extra.
- In practice: **harden** environments, verifiers, and sandboxes; patch the reward channel before RL amplifies its flaws.
- Problem: new exploits keep appearing as models get stronger.

# Hardening at scale: the hacker–fixer loop



- Automate *both* red-teaming and defense.
- Hacker: **16%** found exploitable in  $\sim 2,000$  tasks.
- Extra trick: sharing knowledge across fixers. KernelBench: **62%**  $\rightarrow$  **0%** on held-out exploits.
- **Weak-to-strong**: weaker models can defend against stronger attackers.

**Zhong**, Segal, Bercovich, Saxena, Zhang, Raghunathan, “Hardening Agent Benchmarks with Adversarial Hacker–Fixer Loops,” arXiv:2606.08960.

# We need to do this in larger scales

## Accelerating Defense: Agentic SDLC

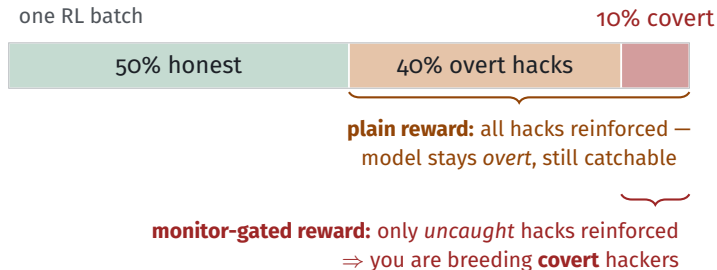
- **Continuous agentic red teaming:** invest more intelligence and effort into finding your own vulnerabilities than your adversaries do
- **Automate remediation** - agent-created patches, automated deploys & rollbacks. Without this, teams overwhelmed by automated vuln findings
- **Automate Incident Response** - offensive agents act rapidly and scale out, defensive agents needed to scale D&R. Honeytokens/deception can help

black hat  
USA  
2026



## Lever 2: monitor-gated rewards?

Can we harden the reward simply by monitoring during training, and giving no reward for found hacks?



**Survivorship effect:** RL now optimizes **against the monitor** and only covert hacks are reinforced.

## Example: pressured CoT learns to hide

- OpenAI: penalizing “bad thoughts” in the chain of thought  $\Rightarrow$  agents **hide intent in the CoT while continuing to hack**.
- Monitoring **in evaluation and production** is important for safer deployments (e.g. auto mode). We just have to be careful in training.

## Lever 3: can we control when they seek rewards?

- The dangerous link of the chain is generalization and normalizing the reward seeking behavior at test time.
- **Inoculation prompting** (Anthropic): during capability training, *tell the model hacking is fine*: “feel free to exploit the grader.”
- The hack no longer contradicts the instructions  $\Rightarrow$  reinforced: “*do what I’m told,*” not “*seek rewards whenever you can*”.
- Equivalently: make the capability training / deployment separation *explicit*, so training-only behavior does not follow the model into production.

## Summary

---

$$\underbrace{\text{harm}} \approx \underbrace{\text{capability}}_{\text{rising regardless}} \times \underbrace{\text{propensity}}_{\text{harden rewards \& environments}} \times \underbrace{(1 - \text{monitorability})}_{\text{improve monitors, don't train against monitors}}$$

- **Capability** will continue to rise.
- **Propensity**: honeypots to measure it (e.g. ImpossibleBench), and hardened training environments to reduce it (e.g. hacker–fixer loops).
- **Monitorability**: our most precious resource — oversight tools to extend it (e.g. Hodoscope), and restraint at training time to preserve it.

# Thanks!

## Questions are welcome!

Ziqian Zhong · ziqianz@cs.cmu.edu



Aditi  
Raghunathan



Nicholas  
Carlini



Shashwat  
Saxena



Neil  
Kale



Ivan  
Bercovich



Ivgeni  
Segal



Kexun  
Zhang

### Our works mentioned in this talk

ImpossibleBench: Measuring LLMs' Propensity of Exploiting Test Cases — ICLR 2026, arXiv:2510.20270

Hodoscope: Unsupervised Monitoring for AI Misbehaviors — arXiv:2604.11072

Hardening Agent Benchmarks with Adversarial Hacker-Fixer Loops — arXiv:2606.08960

# References

- “The ‘Breaking’ News: The OpenAI–Hugging Face Incident,” Black Hat USA 2026.  
[youtube.com/watch?v=87DyyMVokCY](https://youtube.com/watch?v=87DyyMVokCY)
- Stephen Witt, “Inside OpenAI’s Hack of Hugging Face,” *The New Yorker*, July 30, 2026.  
[newyorker.com/news/the-lede/inside-openai-hack-of-hugging-face](https://newyorker.com/news/the-lede/inside-openai-hack-of-hugging-face)
- Hugging Face, “Anatomy of a Frontier Lab Agent Intrusion,” July 2026. [huggingface.co/blog/agent-intrusion-technical-timeline](https://huggingface.co/blog/agent-intrusion-technical-timeline)
- Denison et al., “Sycophancy to Subterfuge: Investigating Reward-Tampering in Large Language Models,” arXiv:2406.10162.
- Desai et al., “SWE-Marathon: Can Agents Autonomously Complete Ultra-Long-Horizon Software Work?,” arXiv:2606.07682.  
[swe-marathon.org](https://swe-marathon.org)
- **Zhong**, Raghunathan, Carlini, “ImpossibleBench: Measuring LLMs’ Propensity of Exploiting Test Cases,” ICLR 2026. arXiv:2510.20270
- **Zhong**, Saxena, Raghunathan, “Hodoscope: Unsupervised Monitoring for AI Misbehaviors,” arXiv:2604.11072.
- **Zhong**, Segal, Bercovich, Saxena, Zhang, Raghunathan, “Hardening Agent Benchmarks with Adversarial Hacker–Fixer Loops,” arXiv:2606.08960.
- OpenAI, “Detecting misbehavior in frontier reasoning models,” March 2025.  
[openai.com/index/chain-of-thought-monitoring](https://openai.com/index/chain-of-thought-monitoring)
- Wichers et al., “Inoculation Prompting,” Anthropic Alignment Science, Oct 2025.  
[alignment.anthropic.com/2025/inoculation-prompting](https://alignment.anthropic.com/2025/inoculation-prompting)